BAHRIA UNIVERSITY

Department of Computer Science

CEN 323 — Computer Organization & Assembly Language

SEMESTER PROJECT

PHASE 2: FINAL IMPLEMENTATION & DOCUMENTATION

Secure Message Embedding using LSB Steganography and XOR Encryption in 8086 Assembly Language

---

Field	Detail
Course Code	CEN 323
Course Instructor	Sir Adnan Jelani
Submission Date	17 May 2026

Submitted by:

Name	Reg. No.	GitHub
Syed Areeb Kareem	01-135232-095	syedareebkareem
Mujeeb Ur Rehman	01-135232-077	MUJEEBBANGASH
Mahrukh Jamal	01-135232-039	Mahrukh1426

GitHub: `https://github.com/syedareebkareem/Coal-Project`

Overleaf: `https://www.overleaf.com/read/sbfwfpnzyskn`

1. Abstract

This project implements a secure message embedding system in 8086 Assembly Language using two complementary techniques: XOR-based encryption and Least Significant Bit (LSB) steganography. The program accepts a plaintext message from the user along with a dynamic password, encrypts the message character by character using the XOR cipher, and then conceals the resulting encrypted data within a simulated pixel array by modifying only the least significant bit of each array element. On the receiver side, the program extracts the hidden bits from the pixel array, reconstructs the encrypted characters, and decrypts them using the same XOR key to recover the original message. The system is menu-driven, allowing the user to send, receive, or view the original message through a 4-option interface. The project demonstrates low-level bit manipulation, stack-based parameter passing, modular procedure design, and DOS interrupt usage — all implemented strictly within the 8086 instruction set using the EMU8086 simulator.

2. Introduction

2.1 Problem Statement

In modern digital communication, transmitting sensitive information securely is a fundamental requirement. Cryptography alone can signal the presence of hidden data, making steganography a valuable complementary technique. This project addresses the problem of implementing both encryption and steganography at the hardware level, using only the limited instruction set and resources available in the 8086 processor architecture.

2.2 Motivation

Assembly language programming provides direct access to hardware resources and forces programmers to think at the level of bits and bytes. Implementing steganography and encryption in assembly reinforces understanding of data representation, memory layout, bitwise operations, and low-level I/O handling — all central to the CEN 323 course objectives. The project also demonstrates that meaningful security operations can be performed within severe hardware constraints.

2.3 Scope

The project is scoped to messages of maximum eight characters, using a simulated 64-element pixel array to store the embedded data. The encryption key is entered dynamically by the user at runtime. The program runs entirely in real mode using DOS interrupts and is tested in the EMU8086 simulator.

2.4 Objectives

- Implement a functional LSB steganography system in 8086 assembly language
- Apply XOR-based encryption and decryption using a user-supplied password
- Demonstrate modular program design using separate procedures for each operation
- Satisfy all Complex Computing Problem (CCA) attributes as defined in the Phase 2 rubric
- Maintain meaningful GitHub commit history showing distributed team contribution

3. System Design

3.1 Architecture Overview

The system is structured as a menu-driven application built around six core procedures. The main procedure initializes the data segment and enters a loop that repeatedly calls `displayMenu` and branches to the selected operation. The Send Cipher path calls `getInput`, `xorEncrypt`, `hideMessage`, and `printArray` in sequence. The Receive Cipher path calls `extractMessage` and `xorDecrypt`. The Print Original path displays the raw input directly from `messageBuffer+2`. The dynamic password is passed to `xorEncrypt` and `xorDecrypt` via the stack using BP as a frame pointer.

3.2 Module Breakdown

Module	Procedure	Owner	Responsibility
Input	getInput	Mujeeb Ur Rehman	Takes user message via buffered DOS input (INT 21h / 0Ah)
Encryption	xorEncrypt	Mujeeb Ur Rehman	XORs each message character with corresponding password byte via stack parameters
Steganography	hideMessage	Syed Areeb Kareem	Embeds encrypted bits into pixel array LSBs
Extraction	extractMessage	Syed Areeb Kareem	Recovers bits from pixel array and reconstructs characters
Decryption	xorDecrypt	Mujeeb Ur Rehman	XORs extracted characters with password via stack to recover original message
Display	displayMenu, printArray, printNumber	Mahrukh Jamal	Prints title, menu, pixel values, and recovered message

3.3 Memory Layout

All data is stored in the data segment. Key variables are:

- `messageBuffer` (10 bytes) — DOS buffered input format: [maxLen, actualLen, chars...]
- `encryptedText` (11 bytes) — encrypted characters after XOR operation
- `extractedBuffer` (11 bytes) — bits recovered from the pixel array
- `finalMessage` (11 bytes) — decrypted output displayed to the user
- `pixelArray` (64 bytes) — simulated pixels initialized to 200; LSBs modified to embed data
- `keyBuffer` (11 bytes) — user-supplied dynamic encryption password
- `messageLength` (1 byte) — count of characters entered by user

3.4 Data Flow

The program runs through a menu-driven loop. On startup, `displayMenu` shows four options. If the user selects Send Cipher (1): `getInput` stores the message → user types

password → password address and length are pushed onto the stack → `xorEncrypt` reads parameters via BP and outputs to `encryptedText` → `hideMessage` modifies pixel array LSBs → `printArray` displays pixel values. If the user selects Receive Cipher (2): `extractMessage` reads pixel array → `xorDecrypt` reads via BP and outputs to `finalMessage` → recovered message displayed. Option 3 prints original message. Option 4 exits.

4. Implementation Details

4.1 getInput Procedure (Mujeeb Ur Rehman)

This procedure displays a prompt and uses DOS function `0Ah` (buffered keyboard input) to read the user message into `messageBuffer`. Byte 0 holds the maximum length, byte 1 is filled by DOS with actual characters typed, and bytes 2 onwards contain the actual data. The actual length at `messageBuffer+1` is copied into `messageLength` for use by subsequent procedures.

```
lea dx, messageBuffer     ; point to buffer
mov ah, 0ah               ; buffered input function
int 21h                   ; DOS fills buffer
mov al, messageBuffer+1   ; actual length typed
mov messageLength, al     ; save for later use
```

4.2 xorEncrypt Procedure (Mujeeb Ur Rehman)

Before calling this procedure, the main program pushes the address of `keyBuffer+2` and the message length onto the stack. Inside the procedure, BP is used as a frame pointer (`PUSH BP / MOV BP,SP`) and parameters are retrieved using `[BP+6]` for the password address and `[BP+4]` for the count. BX is loaded with the password address and CX with the length. SI points to `messageBuffer+2` and DI points to `encryptedText`. Both SI and BX advance together so each message character is XORed with the corresponding password character. All registers are saved at entry and restored at exit. The procedure returns with `RET 4` to discard the two stack parameters.

```
mov bx, [bp+6]    ; password address from stack
mov cx, [bp+4]    ; character count from stack
mov al, [si]      ; current message character
mov dl, [bx]      ; corresponding password character
xor al, dl        ; encrypt: msg XOR key
mov [di], al      ; store encrypted result
```

4.3 hideMessage Procedure (Syed Areeb Kareem)

This is the core steganography procedure. It iterates over each encrypted character and embeds its 8 bits into 8 consecutive elements of the pixel array. For each bit, the procedure clears the LSB of the current pixel using `AND AL, 11111110B`, extracts the current bit of the character using `AND BH, 00000001B`, and inserts it into the pixel using `OR AL, BH`. The character bits are processed LSB-first using `SHR BL,1` after each bit.

```
and al, 11111110b  ; clear pixel LSB
mov bh, bl         ; copy character byte
and bh, 00000001b  ; isolate current bit
or  al, bh         ; insert bit into pixel
shr bl, 1          ; shift to next bit
```

4.4 extractMessage Procedure (Syed Areeb Kareem)

This procedure reverses the embedding process. For each character, it reads 8 consecutive pixel elements, extracts the LSB of each, and reconstructs the original byte. A mask register BH starts at 1 and shifts left after each bit is recovered. When the extracted LSB is 1, it is ORed into BL at the position indicated by BH. When all 8 bits are processed, BL contains the extracted encrypted character stored into `extractedBuffer`.

```
and al, 00000001b  ; isolate pixel LSB
cmp al, 1
jnz defaultZero
or  bl, bh         ; set bit in character
defaultZero:
shl bh, 1          ; shift mask left
```

4.5 xorDecrypt Procedure (Mujeeb Ur Rehman)

Structurally identical to `xorEncrypt` but reads from `extractedBuffer` and writes to `finalMessage`. The same stack-based parameter passing is used. The same XOR operation with the same password bytes undoes the encryption character by character, recovering the original plaintext. The procedure uses `RET 4` to clean up the stack parameters.

4.6 Display Procedures (Mahrukh Jamal)

`displayMenu` shows the program title and 4-option menu using INT 21h / 09h. `printArray` calculates total pixels to display by multiplying `messageLength` by 8 using `MUL BL`, then loops through the pixel array calling `printNumber` for each element. `printNumber` converts an 8-bit value (0–255) to three ASCII decimal digits using successive division by 100 and 10, printing each digit with INT 21h / 02h.

4.7 DOS Interrupts Used

Interrupt	Function (AH)	Purpose
INT 21h	01h	Read single character from keyboard
INT 21h	02h	Display single character in DL
INT 21h	09h	Display dollar-terminated string pointed to by DX
INT 21h	0Ah	Buffered keyboard input into structured buffer
INT 21h	4Ch	Terminate program and return to operating system

5. CCA Attribute Mapping

CCA1 — Complex Problem-Solving

This project addresses a non-trivial computing problem requiring multi-stage processing and interaction of six distinct modules. The steganography module alone requires two nested loops per character: an outer loop iterating over each of the eight message characters and an inner loop processing all eight bits of each character individually. The extraction procedure must exactly reverse this bit-level process using a shifting mask register across both loops. The encryption layer adds a further dependency: the output of `xorEncrypt` becomes the input of `hideMessage`, and the output of `extractMessage` becomes the input of `xorDecrypt`. Any misalignment in buffer pointers or loop counters at any stage causes the entire pipeline to fail, making this a genuinely complex program to design, debug, and verify.

CCA4 — Design Constraints

The entire project is implemented within the strict constraints of the 8086 instruction set architecture. General-purpose registers are limited to AX, BX, CX, DX, SI, and DI, each with specific usage conventions. The LOOP instruction requires CX as its counter, so DL was used as a secondary counter for the inner bit loop to avoid conflicts. The 8086 has no

floating-point instructions, so all number printing uses integer division (`DIV BL`). There is no dynamic memory allocation; all buffers are statically declared in the data segment. Parameter passing to `xorEncrypt` and `xorDecrypt` is done through the stack using BP as a frame pointer — the caller pushes the password address and length, the callee retrieves them at `[BP+6]` and `[BP+4]`, then cleans up with `RET 4`. Every design decision is a direct consequence of these architectural constraints.

CCA8 — Team Collaboration

The project was divided into three distinct modules each owned by a different team member. Mujeeb Ur Rehman was responsible for `getInput`, `xorEncrypt`, and `xorDecrypt` — the input and encryption pipeline. Syed Areeb Kareem implemented `hideMessage` and `extractMessage` — the steganography core. Mahrukh Jamal handled `displayMenu`, `printArray`, and `printNumber` — all display and user interface logic, including the menu-driven control flow in `main`. Collaboration is evidenced by the distributed commit history on the GitHub repository at https://github.com/syedareebkareem/Coal-Project, where each member committed their own procedures with descriptive messages across the development period.

6. Testing & Results

6.1 Test Cases

#	Input	Password	Expected	Actual	Result
1	hi	abc	hi	hi	PASS
2	areeb	key12	areeb	areeb	PASS
3	MUJEEB	pass1	MUJEEB	MUJEEB	PASS
4	A	z	A	A	PASS
5	12345678	abcdefgh	12345678	12345678	PASS

6.2 Observations

In all test cases the message was correctly encrypted, embedded into the pixel array, extracted, and decrypted back to the original. The pixel array before embedding shows all elements as 200 (11001000 in binary). After embedding, elements either retain 200 or change to 201 (11001001) depending on whether the embedded bit is 0 or 1, confirming that only the LSB is modified and the rest of the pixel value is preserved. The `printNumber` procedure correctly handles all values in the range 0–255.

7. Team Contributions

Member Name	Reg. No.	Module(s) Owned	Key Procedures
Syed Areeb Kareem	01-135232-095	Steganography & Extraction	hideMessage, extractMessage
Mujeeb Ur Rehman	01-135232-077	Input & Encryption / Decryption	getInput, xorEncrypt, xorDecrypt
Mahrukh Jamal	01-135232-039	UI, Display & Flow Control	displayMenu, printArray, printNumber, main

All three members contributed commits to the GitHub repository. Commit messages reflect the specific procedure added or modified in each commit. Integration was performed after individual modules were verified independently.

8. Challenges & Lessons Learned

8.1 Technical Challenges

- **Register conflicts:** Both loops needed a counter but LOOP uses CX exclusively. Resolved by using DL as the inner bit counter with `DEC DL / JNZ`, reserving CX only for the outer LOOP.
- **Buffer offset confusion:** DOS buffered input fills the buffer starting at offset +2, not +0. Reading from +0 returned the max length byte rather than actual characters. Fixed by loading SI with `messageBuffer+2` and `keyBuffer+2`.
- **Label name conflicts:** `extractMessage` reused the label names `characterLoop` and `bitLoop` already defined in `hideMessage`. EMU8086 reported duplicate label errors. Resolved by renaming to `extCharLoop` and `extBitLoop`.
- **Dynamic key implementation:** Switching from a fixed XOR key (05h) to a dynamic user-supplied password required restructuring how the key was accessed, using stack-based parameter passing with BP as a frame pointer.
- **printNumber overflow:** Values above 99 produced incorrect output because AH was not cleared before DIV, causing division to operate on full AX. Fixed by adding `MOV AH, 0` before each division.

8.2 Lessons Learned

- Working within strict register constraints requires careful planning before writing any code.
- Bit manipulation in assembly is precise and unforgiving — a single incorrect AND mask shifts all subsequent bits to wrong positions.
- Modular procedure design simplified debugging by isolating problems to individual procedures.
- GitHub commit discipline matters — committing each procedure separately with a clear message makes contribution tracking straightforward.

9. Conclusion

This project successfully implemented a working secure message embedding system in 8086 Assembly Language. The program correctly encrypts a user-supplied message using a dynamic XOR cipher, hides the encrypted data within a simulated pixel array using LSB steganography, extracts and decrypts the hidden message on the receiver side, and displays results through a menu-driven interface. All six procedures function correctly within the constraints of the 8086 instruction set and the EMU8086 simulator.

9.1 Limitations

- The message is limited to eight characters due to the fixed 64-element pixel array.
- The XOR cipher is not cryptographically secure against known-plaintext attacks.
- No validation checks whether the password length is at least equal to the message length.

9.2 Future Work

- Extend the pixel array to support longer messages.
- Replace XOR with a stronger cipher such as a Vigenere implementation in assembly.
- Add input validation for password and message length compatibility.
- Implement file I/O to save and load the pixel array, simulating real image files.

10. References

[1] Intel Corporation, *8086 Family User's Manual*, Intel Corp., Santa Clara, CA, 1979.

[2] P. Abel, *IBM PC Assembly Language and Programming*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2001.

[3] R. Shetty, "LSB Image Steganography," ResearchGate, 2014. [Online]. Available: https://www.researchgate.net

[4] Ralf Brown, "Interrupt List," 1999. [Online]. Available: http://www.ctyme.com/rbrown.htm

[5] S. A. Kareem, M. U. Rehman, and M. Jamal, "Coal-Project GitHub Repository," GitHub, 2026. [Online]. Available: https://github.com/syedareebkareem/Coal-Project

[6] Overleaf Project. [Online]. Available: https://www.overleaf.com/read/sbfwfpnzyskn#63c1c6

AI Disclosure: Claude (Anthropic) was used to assist in structuring and drafting sections of this report. All code was written by the team members independently.